La computer vision dietro *Rectangular Filter*

Mani, rettangoli, filtri e come far sparire una persona

Documentazione tecnica del progetto

Novembre 2026

Sommario

Rectangular Filter è una web app che, a partire dal flusso della fotocamera, riconosce le mani, costruisce un quadrilatero con le punte di pollici e indici e applica un filtro solo al suo interno. Un gesto (pugno seguito da una V) fa sparire la persona dalla scena, lasciando visibile solo lo sfondo. Questo documento spiega le tecniche di computer vision e di elaborazione delle immagini usate a ogni passo: le reti neurali di MediaPipe per mani e sagoma, la geometria del quadrilatero, il filtro One Euro, le funzioni di distanza con segno, i filtri come operatori sull'immagine, la classificazione delle pose della mano e il compositing con sottrazione dello sfondo. Tutti i numeri citati sono quelli effettivamente usati nel codice.

Indice

1 La pipeline in un colpo d'occhio **2**

2 Il fotogramma e i sistemi di coordinate **2**

3 Trovare le mani: MediaPipe Hands **2**
3.1 Primo stadio: rilevare il palmo . . . 2
3.2 Secondo stadio: i 21 punti . . . 3
3.3 Tracciamento nel tempo . . . 3

4 Dai punti al quadrilatero **4**

5 Stabilizzare gli angoli: il filtro One Euro **4**

6 Dal quadrilatero alla maschera: distanza con segno **5**

7 I filtri come operatori sull'immagine **5**
7.1 Sfocatura con la spirale di Vogel . . . 6
7.2 Caleidoscopio . . . 7

8 Riconoscere i gesti: pugno e V **7**
8.1 Le caratteristiche di ciascun dito . . . 7
8.2 Le regole . . . 7
8.3 La sequenza nel tempo . . . 7

9 Sparire: segmentazione, sottrazione dello sfondo e compositing **8**
9.1 Che cosa c'era dietro: la lastra di sfondo . . . 8
9.2 Quali pixel sono tuoi: due indizi complementari . . . 8
9.3 Costi asimmetrici: meglio coprire troppo . . . 9
9.4 La transizione: sfocatura e dissolvenza . . . 9
9.5 Limiti . . . 10

10 Tempo reale e privacy **10**

1 La pipeline in un colpo d'occhio

Ogni volta che la fotocamera produce un nuovo fotogramma (tipicamente 30 al secondo), l'app esegue la catena di figura 1. Tutto avviene sul dispositivo: le reti neurali girano nel browser (WebAssembly o WebGL) e il compositing finale è uno *shader* eseguito dalla GPU, un pixel alla volta e in parallelo.

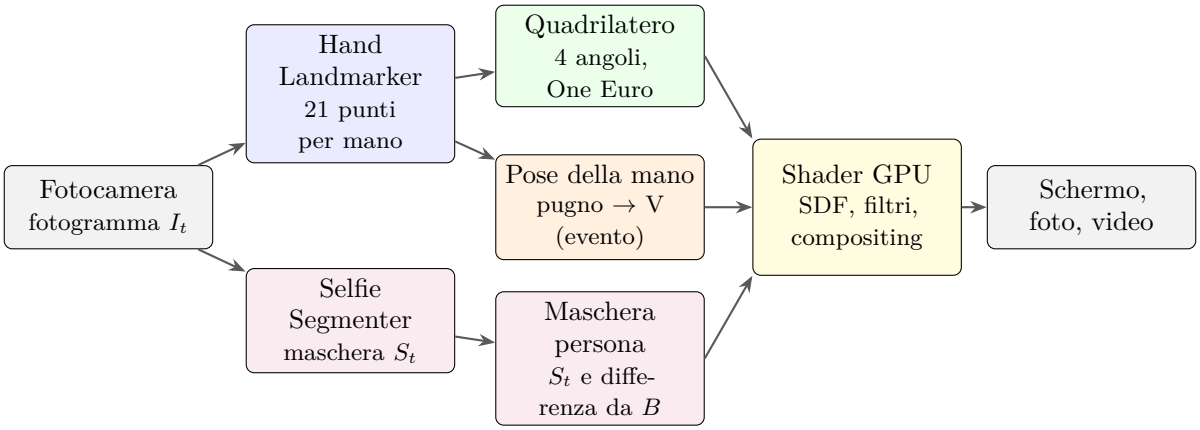


Figura 1: Flusso di un fotogramma. La segmentazione viene eseguita solo mentre l'effetto "sparizione" è attivo, per risparmiare calcolo.

Il ciclo principale è guidato da `requestAnimationFrame`. Il riconoscimento delle mani viene eseguito solo quando il video ha davvero un fotogramma nuovo: sugli schermi a 60 o 120 Hz si evita di analizzare due volte la stessa immagine.

2 Il fotogramma e i sistemi di coordinate

Un fotogramma è una funzione $I : \Omega \to [0,1]^3$ che associa a ogni pixel un colore RGB. Le reti di MediaPipe restituiscono punti in *coordinate normalizzate* $(u, v) \in [0,1]^2$ rispetto al fotogramma originale di larghezza w e altezza h, con v che cresce verso il basso.

Lo schermo (larghezza W, altezza H) ha in genere un'altra proporzione, quindi il video viene scalato "a copertura" (*cover*), cioè riempie lo schermo e taglia ciò che avanza:

$$s = \max\!\left(\frac{W}{w}, \frac{H}{h}\right), \qquad \mathbf{o} = \tfrac{1}{2}(W - sw,\ H - sh), \qquad \mathbf{p} = \mathbf{o} + s\,(w\,u',\ h\,v), \tag{1}$$

dove $u' = 1 - u$ se si usa la fotocamera frontale (effetto specchio, come in un selfie) e $u' = u$ altrimenti. Nello shader si usa la trasformazione inversa: per ogni pixel $\mathbf{p}$ dello schermo si calcola quale punto (u, v) del video campionare.

3 Trovare le mani: MediaPipe Hands

Per le mani l'app usa *MediaPipe Hand Landmarker* [1, 2], una pipeline di due reti neurali convoluzionali progettata per girare in tempo reale anche sui telefoni.

3.1 Primo stadio: rilevare il palmo

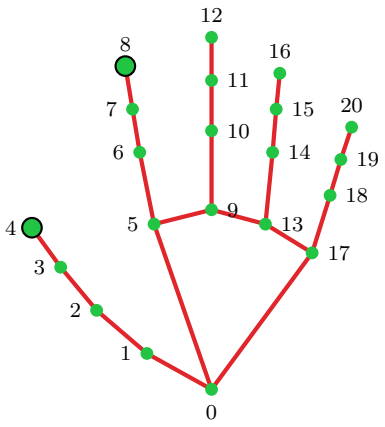
Una mano è un oggetto articolato: le dita cambiano completamente la sua forma e il suo riquadro. Il palmo, invece, è quasi rigido. Per questo il primo stadio (*BlazePalm*) non cerca la mano intera ma il palmo (o il pugno). È un rilevatore a colpo singolo nello stile di SSD [4] e BlazeFace [3]. Una rete *encoder–decoder* produce mappe di caratteristiche a più scale. Per ogni *ancora* (un riquadro

di riferimento) la rete stima la probabilità che contenga un palmo e la correzione da applicare al riquadro, insieme ad alcuni punti chiave. Siccome i palmi sono circa quadrati, bastano ancore quadrate, e questo riduce molto il numero di ancore. Per l'addestramento si usa la *focal loss* [5], che pesa di più gli esempi difficili. Infine la *non-maximum suppression* elimina i riquadri duplicati.

Dal palmo trovato si ricava un riquadro *ruotato*: la rotazione allinea il segmento tra polso e base del dito medio con l'asse verticale. Il riquadro viene poi allargato abbastanza da contenere le dita.

3.2 Secondo stadio: i 21 punti

Il ritaglio ruotato (224×224 pixel) passa a una seconda rete che *regredisce* direttamente le coordinate di 21 punti anatomici (figura 2), più un punteggio di presenza della mano e la sua lateralità (destra o sinistra). Ogni punto ha coordinate (x, y, z): x e y sono normalizzate all'immagine, mentre z è una profondità relativa con origine nel polso, nella stessa scala di x (valori minori indicano punti più vicini alla fotocamera). La rete è addestrata su decine di migliaia di foto reali annotate a mano e su immagini sintetiche di un modello 3D della mano, in pose e sfondi molto vari.



0 polso
1–4 pollice (CMC, MCP, IP, punta)
5–8 indice (MCP, PIP, DIP, punta)
9–12 medio
13–16 anulare
17–20 mignolo

Cerchiati: punte di pollice (4) e indice (8), cioè gli angoli del rettangolo.

Figura 2: I 21 punti della mano restituiti da MediaPipe e i collegamenti che l'app disegna (linee rosse, punti verdi, come nel reel di riferimento).

3.3 Tracciamento nel tempo

In modalità `VIDEO` la pipeline non riesegue ogni volta il rilevatore di palmi, che è il passo più costoso. Il riquadro della mano al fotogramma $t+1$ viene ricavato dai punti trovati al fotogramma t, perché la mano si sposta poco tra due fotogrammi consecutivi. Il rilevatore torna in gioco solo quando il punteggio di presenza scende sotto la soglia (mano persa) o quando ci sono meno mani del massimo richiesto (qui 2). Per questo a ogni chiamata va passato un *timestamp* strettamente crescente.

GPU o CPU. La stessa rete può essere eseguita con il *delegate* GPU (WebGL) o CPU (WebAssembly con istruzioni SIMD tramite XNNPACK). L'app prova prima la GPU e passa alla CPU in caso di errore. L'utente può anche forzare la CPU dalle impostazioni.

Lateralità. MediaPipe assume che l'immagine in ingresso sia specchiata, come un selfie. L'app invece passa il fotogramma non specchiato, quindi le etichette "destra" e "sinistra" risultano scambiate. Per questo vengono usate solo come identificatore stabile di ciascuna mano (§8) e mai mostrate all'utente.

4 Dai punti al quadrilatero

Siano $\mathbf{a}_k$ e $\mathbf{b}_k$ i punti k delle due mani, già convertiti in pixel dello schermo. Le mani vengono ordinate in base alla posizione orizzontale del polso: $\mathbf{a}$ è quella più a sinistra sullo schermo.

Due mani. Come nel reel, il quadrilatero è $Q = (\mathbf{a}_8, \mathbf{b}_8, \mathbf{b}_4, \mathbf{a}_4)$: gli indici sono gli angoli in alto e i pollici quelli in basso.

Una mano. Con punta del pollice $\mathbf{t} = \mathbf{p}_4$ e punta dell'indice $\mathbf{i} = \mathbf{p}_8$, il rettangolo è quello allineato agli assi che ha $\mathbf{t}$ e $\mathbf{i}$ come vertici opposti: $[\min(t_x, i_x), \max(t_x, i_x)] \times [\min(t_y, i_y), \max(t_y, i_y)]$.

Le mani che stanno facendo il gesto pugno o V (§8) vengono escluse, così il gesto non disegna rettangoli per sbaglio.

Quando mostrarlo. Se le dita si chiudono il quadrilatero degenera. L'area viene calcolata come somma delle aree (in valore assoluto) dei due triangoli ABC e ACD:

$$\mathcal{A}(Q) = \tfrac{1}{2}\Big(\big|(B-A)\times(C-A)\big| + \big|(C-A)\times(D-A)\big|\Big), \tag{2}$$

dove $\times$ indica il prodotto vettoriale 2D, $(x_1, y_1)\times(x_2, y_2) = x_1y_2 - y_1x_2$. A differenza della formula di Gauss (*shoelace*), questa somma non si annulla quando il quadrilatero si incrocia "a farfalla". L'effetto si mostra solo se $\mathcal{A}(Q) > 0{,}0015\,WH$, cioè se occupa più dello 0,15% dello schermo.

L'opacità α dell'effetto segue un'esponenziale con costante di tempo $\tau = 90\,\text{ms}$, e il quadrilatero resta visibile per 160 ms dopo l'ultima rilevazione valida:

$$\alpha \leftarrow \alpha + (\alpha^\star - \alpha)(1 - e^{-\Delta t/\tau}), \qquad \alpha^\star \in \{0, 1\}. \tag{3}$$

In questo modo l'effetto entra ed esce in dissolvenza e non sfarfalla se la mano viene persa per un solo fotogramma.

5 Stabilizzare gli angoli: il filtro One Euro

I punti stimati da una rete tremano di qualche pixel da un fotogramma all'altro (*jitter*). Un filtro passa-basso li calma, ma introduce ritardo (*lag*): il rettangolo "insegue" le dita. Il filtro One Euro [7] risolve il compromesso adattando la frequenza di taglio alla velocità del movimento. Per ognuna delle 8 coordinate (4 angoli × 2 assi):

$$\hat{x}_i = a_i\, x_i + (1 - a_i)\,\hat{x}_{i-1}, \qquad a_i = \frac{1}{1 + \tau_i/\Delta t}, \qquad \tau_i = \frac{1}{2\pi f_{c,i}}, \tag{4}$$

$$f_{c,i} = f_{\min} + \beta\,|\hat{\dot{x}}_i|, \tag{5}$$

dove $\hat{\dot{x}}_i$ è la velocità stimata (anch'essa filtrata, con taglio fisso a 1 Hz).

Quando la mano è ferma la velocità è quasi nulla, il taglio resta basso ($f_{\min}$) e il tremolio viene eliminato. Quando la mano si muove in fretta, f_c sale e il filtro "si apre", azzerando il ritardo. Il cursore *Stabilizzazione* $s \in \{0, \ldots, 10\}$ dell'app imposta

$$f_{\min} = \frac{6}{1 + 0{,}9\,s}\ \text{Hz}, \qquad \beta = 0{,}004 + 0{,}0012\,(10 - s)\ \text{px}^{-1}. \tag{6}$$

Con $s = 0$ il filtro è di fatto disattivato. Quando il quadrilatero ricompare dopo essere sparito, il filtro viene azzerato, così gli angoli non scivolano dalla posizione vecchia.

6 Dal quadrilatero alla maschera: distanza con segno

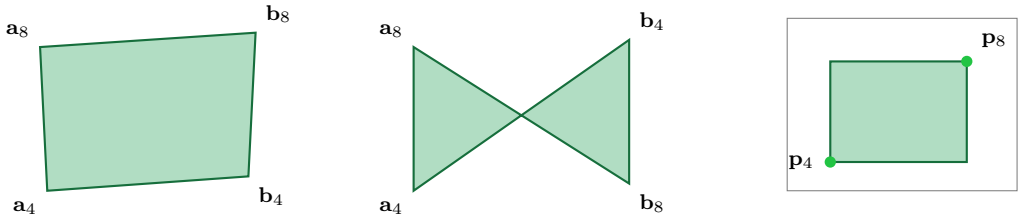
Lo shader deve decidere, per ciascuno dei milioni di pixel dello schermo, se il pixel sta dentro il quadrilatero e quanto è vicino al bordo. Lo fa con una *funzione di distanza con segno* (SDF) [8], negativa dentro e positiva fuori.

Distanza dai lati. Per un segmento $\overline{\mathbf{ab}}$ si proietta il pixel $\mathbf{p}$ sulla retta e si limita la proiezione al segmento:

$$h = \mathrm{clamp}\left(\frac{(\mathbf{p}-\mathbf{a})\cdot(\mathbf{b}-\mathbf{a})}{\|\mathbf{b}-\mathbf{a}\|^2},\, 0,\, 1\right), \qquad d_{ab}(\mathbf{p}) = \|\mathbf{p}-\mathbf{a}-h(\mathbf{b}-\mathbf{a})\|. \tag{7}$$

La distanza dal quadrilatero è il minimo sui quattro lati: $d(\mathbf{p}) = \min d_{ab}(\mathbf{p})$.

Il segno: regola pari-dispari. Si immagina un raggio orizzontale che parte da $\mathbf{p}$ e si contano i lati che attraversa. Un numero dispari di attraversamenti significa che il pixel è dentro. Il segno è quindi $\sigma = (-1)^n$ e $\mathrm{sd}(\mathbf{p}) = -\sigma\, d(\mathbf{p})$. Questa regola funziona anche quando i lati si incrociano: un quadrilatero "a farfalla" diventa semplicemente due triangoli pieni (figura 3).



mani "a cornice" — mani incrociate: "farfalla" — una mano

Figura 3: Le tre forme possibili. Con la regola pari-dispari il caso incrociato non richiede nessun trattamento speciale.

Bordi morbidi. Con la funzione *smoothstep*

$$\mathrm{S}(e_0, e_1, x) = 3t^2 - 2t^3, \qquad t = \mathrm{clamp}\left(\frac{x-e_0}{e_1-e_0}, 0, 1\right), \tag{8}$$

la maschera vale $m(\mathbf{p}) = 1 - \mathrm{S}(-f,\, f,\, \mathrm{sd}(\mathbf{p}))$, dove f è la sfumatura scelta dall'utente più 0,75 px. Quei 0,75 px fanno da *anti-aliasing*: il bordo non è mai seghettato, nemmeno con sfumatura zero. Se il filtro va applicato "fuori" si usa $1 - m$. Il colore finale è un'interpolazione lineare, cioè il caso più semplice del compositing alfa [9]:

$$O(\mathbf{p}) = (1 - \alpha\, m(\mathbf{p}))\, I(\mathbf{p}) + \alpha\, m(\mathbf{p})\, F[I](\mathbf{p}), \tag{9}$$

dove $F[I]$ è l'immagine filtrata. Il bordo opzionale è una striscia dove $|\mathrm{sd}(\mathbf{p})|$ è minore di metà spessore.

Perché sulla GPU. Il calcolo di ogni pixel è indipendente dagli altri: è il caso ideale per un *fragment shader*. Un iPad esegue queste operazioni su circa 2–4 milioni di pixel per fotogramma senza impegnare la CPU, che resta libera per le reti neurali.

7 I filtri come operatori sull'immagine

Ogni filtro è una funzione $F[I]$ calcolata nello shader. Alcuni sono puntuali (dipendono solo dal colore del pixel), altri usano un intorno del pixel (convoluzioni) o deformano le coordinate.

Luminanza. Molti filtri partono dalla luminanza, che pesa i canali in base alla sensibilità dell'occhio (raccomandazione ITU-R BT.601 [10]): $Y = 0{,}299\,R + 0{,}587\,G + 0{,}114\,B$.

Gradiente di Sobel. I contorni sono i punti in cui la luminanza cambia bruscamente. L'operatore di Sobel [11, 12] stima le derivate con due convoluzioni 3×3:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * Y, \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * Y, \qquad e = \sqrt{G_x^2 + G_y^2}, \quad \theta = \operatorname{atan2}(G_y, G_x). \tag{10}$$

Filtro	**Che cosa calcola**
Acido	Mappa luminanza e contorni su un arcobaleno che scorre nel tempo: $h = \operatorname{frac}(2{,}6\,Y + 1{,}4\,e + 0{,}1\,n - 0{,}12\,t)$, colore = HSV($h$, 0,85, v), dove n è un rumore pseudo-casuale (grana) e t il tempo. È il look del reel.
Bianco e nero	$Y' = \operatorname{clamp}(1{,}15\,(Y - 0{,}5) + 0{,}5)$: scala di grigi con un leggero aumento di contrasto.
Invisibile	Al posto del video mostra lo sfondo catturato B (vedi §9).
Termico	*Falsi colori*: la luminanza (leggermente sfocata) indicizza una tavolozza lineare a tratti nero → viola → rosso → arancio → bianco, come una termocamera.
Negativo	$F[I] = 1 - I$.
Pixel	Quantizza lo spazio: $\mathbf{q} = (\lfloor \mathbf{p}/b \rfloor + \frac{1}{2})\,b$ con blocchi di $b = 22$ px (riferiti a 720 p), e quantizza i colori a 6 livelli. È un sottocampionamento senza filtro anti-alias: l'*aliasing* diventa un effetto estetico.
Neon	Contorni su nero, S(0,08, 0,5, e), con tinta data dall'orientazione θ del gradiente: ogni direzione ha il suo colore.
Glitch	Aberrazione cromatica: $R(u + \delta)$, $G(u)$, $B(u - \delta)$ con δ che oscilla, più strappi orizzontali casuali a bande e linee di scansione.
Fumetto	Posterizzazione a 4 livelli, saturazione aumentata, contorni neri da Sobel e un *retino* a punti il cui raggio cresce con $1 - Y$ (stampa a mezzatinta).
Sfocato	Media di 25 campioni disposti a spirale di Vogel (§7.1).
Raggi X	Luminanza invertita con correzione gamma, tinta azzurra e contorni luminosi.
Caleidoscopio	Coordinate polari attorno al centro del quadrilatero e "ripiegamento" dell'angolo: simmetria a 8 settori.

Tabella 1: I 12 filtri e l'operazione di elaborazione d'immagine che li realizza.

7.1 Sfocatura con la spirale di Vogel

Una sfocatura gaussiana esatta richiederebbe molti campioni. Lo shader usa invece $N = 24$ campioni (più il centro) distribuiti in modo uniforme in un disco con la *spirale di Vogel* [13], la stessa disposizione dei semi del girasole:

$$r_k = R\sqrt{k/N}, \qquad \varphi_k = k\,\varphi_g, \qquad \varphi_g = \pi(3 - \sqrt{5}) \approx 2{,}39996, \tag{11}$$

dove φ_g è l'*angolo aureo*. La radice quadrata rende uniforme la densità sull'area e l'angolo aureo evita che i campioni si allineino. Il risultato è simile a un *bokeh* circolare di raggio R. La stessa funzione serve alla transizione della sparizione, con R variabile.

7.2 Caleidoscopio

Con $\mathbf{c}$ il centro del quadrilatero, $\mathbf{d} = \mathbf{p} - \mathbf{c}$, $r = \|\mathbf{d}\|$ e $\phi = \operatorname{atan2}(d_y, d_x) + \omega t$, l'angolo viene ripiegato in un settore di ampiezza $\Delta = 2\pi/8$:

$$\phi' = \left|(\phi \bmod \Delta) - \tfrac{\Delta}{2}\right|, \qquad \mathbf{p}' = \mathbf{c} + r\,(\cos\phi', \sin\phi'). \tag{12}$$

Campionare il video in $\mathbf{p}'$ produce un'immagine con la simmetria del gruppo diedrale: 8 spicchi, ciascuno speculare al vicino.

8 Riconoscere i gesti: pugno e V

Per far sparire la persona l'app riconosce una *sequenza* di due pose della stessa mano: un pugno e poi una V con indice e medio. La classificazione usa regole geometriche sui 21 punti, non una rete dedicata.

8.1 Le caratteristiche di ciascun dito

I punti vengono portati in uno spazio isotropo, $\mathbf{X} = (x\,w,\ y\,h,\ z\,w)$, in modo che le distanze abbiano la stessa unità su tutti gli assi. Per ogni dito (escluso il pollice), con $\mathbf{M}$ la nocca (MCP), $\mathbf{P}$ l'articolazione intermedia (PIP), $\mathbf{T}$ la punta e $\mathbf{W}$ il polso, si calcolano due misure:

$$\sigma = \cos\angle(\mathbf{P}-\mathbf{M},\ \mathbf{T}-\mathbf{P}) = \frac{(\mathbf{P}-\mathbf{M})\cdot(\mathbf{T}-\mathbf{P})}{\|\mathbf{P}-\mathbf{M}\|\,\|\mathbf{T}-\mathbf{P}\|} \qquad \text{(rettilineità)}, \tag{13}$$

$$\rho = \frac{\|\mathbf{T}-\mathbf{W}\|}{\|\mathbf{P}-\mathbf{W}\|} \qquad \text{(allungamento)}. \tag{14}$$

Un dito disteso ha $\sigma \approx 1$ (le falangi sono allineate) e $\rho \approx 1{,}3$ (la punta è più lontana dal polso della nocca intermedia). In un dito chiuso a pugno la punta torna verso il palmo: $\sigma < 0$ e $\rho < 1$. Entrambe le misure sono rapporti o angoli, quindi non cambiano con la distanza dalla fotocamera, con la posizione o con la rotazione della mano. Siccome usano anche z, funzionano pure quando le dita puntano verso l'obiettivo.

8.2 Le regole

$$\begin{aligned}
\textsf{disteso} &\iff \sigma > 0{,}55 \ \wedge\ \rho > 1{,}08,\\
\textsf{piegato} &\iff \sigma < 0{,}25 \ \vee\ \rho < 0{,}92,\\
\textsf{pugno} &\iff \text{indice, medio, anulare, mignolo tutti } \textsf{piegati},\\
\textsf{V} &\iff \text{indice e medio } \textsf{distesi} \ \wedge\ \text{anulare e mignolo } \textsf{piegati} \ \wedge\ \frac{\|\mathbf{T}_8 - \mathbf{T}_{12}\|}{\|\mathbf{X}_9 - \mathbf{W}\|} > 0{,}3.
\end{aligned}$$

L'ultima condizione richiede che le due dita siano *divaricate*: due dita unite non sono una V. Il pollice viene ignorato, perché in entrambe le pose può stare ovunque. Le regole sono volutamente asimmetriche: per "disteso" servono entrambe le condizioni, per "piegato" ne basta una. Così una V viene riconosciuta solo quando è netta, mentre un pugno un po' lasco conta lo stesso.

8.3 La sequenza nel tempo

Le pose stimate fotogramma per fotogramma sono rumorose, quindi l'app usa una piccola macchina a stati per ogni mano, identificata dalla sua lateralità:

1. **Anti-rimbalzo** (*debounce*): una posa diventa "ufficiale" solo se resta uguale per almeno 70 ms.

2. **Pugno valido**: il pugno deve durare almeno 220 ms. Da quel momento, e per tutto il tempo in cui resta chiuso, l'app aggiorna l'istante t_{pugno}. Sullo schermo compare "ora V".

3. **V**: la V deve iniziare entro 1,6 s dall'ultimo pugno valido e durare almeno 180 ms. A quel punto scatta l'evento.

4. **Pausa** (*cooldown*): per 1,2 s non può scattare un altro evento.

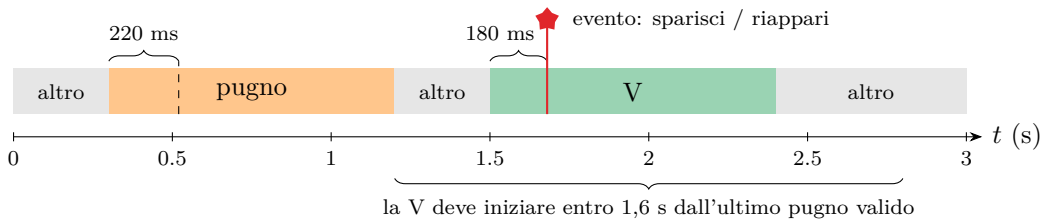


Figura 4: Diagramma temporale della sequenza pugno → V.

Chiedere due pose diverse *in quest'ordine* riduce molto i falsi positivi. Una V da sola, gesto frequente in foto e video, non fa scattare niente. Ogni evento *inverte* lo stato: il primo fa sparire, il secondo fa riapparire.

9 Sparire: segmentazione, sottrazione dello sfondo e compositing

Per "togliere" una persona serve sapere due cose: *quali* pixel le appartengono e *che cosa* c'era dietro.

9.1 Che cosa c'era dietro: la lastra di sfondo

L'app cattura una *lastra pulita* (*clean plate*) B, cioè un fotogramma della scena vuota, durante un conto alla rovescia in cui l'utente esce dall'inquadratura. L'ipotesi è che la fotocamera resti ferma e la luce costante: in quel caso, in ogni pixel non coperto dalla persona, vale $I_t(\mathbf{p}) \approx B(\mathbf{p})$.

9.2 Quali pixel sono tuoi: due indizi complementari

1. Segmentazione semantica. Il *Selfie Segmenter* di MediaPipe è una piccola rete convoluzionale *encoder–decoder* derivata da MobileNetV3 [6]. Riceve l'immagine ridotta a 256×256 e restituisce per ogni pixel la probabilità $S(\mathbf{p}) \in [0, 1]$ che appartenga a una persona. La maschera viene poi riportata alla risoluzione del fotogramma. È una stima "semantica": riconosce una persona anche se ha i vestiti dello stesso colore del muro, ma tende a sbagliare sui contorni (capelli, dita) e ignora le ombre e gli oggetti che la persona tiene in mano.

2. Sottrazione dello sfondo. Il secondo indizio è il più classico della videosorveglianza [14]: ciò che è diverso dalla lastra vuota è primo piano. Per ridurre il rumore del sensore si confrontano medie locali su 5 pixel a croce:

$$D(\mathbf{p}) = \|\bar{I}(\mathbf{p}) - \bar{B}(\mathbf{p})\|, \qquad M_d(\mathbf{p}) = \mathrm{S}(0{,}09,\ 0{,}2,\ D(\mathbf{p})). \tag{15}$$

Questo indizio coglie ombre, oggetti tenuti in mano e i pezzi che la rete non vede. Fallisce però dove la persona ha lo stesso colore dello sfondo (*camouflage*). I punti deboli dei due metodi sono diversi, quindi conviene combinarli.

9.3 Costi asimmetrici: meglio coprire troppo

L'osservazione chiave è che i due errori possibili non costano uguale:

- **falso negativo** (un pixel della persona non viene sostituito): resta un "fantasma" ben visibile;
- **falso positivo** (un pixel dello sfondo viene sostituito): non si vede, perché lì $I \approx B$ e si sostituisce lo sfondo con se stesso.

Per questo tutte le scelte spingono la maschera ad *allargarsi*:

$$S_d(\mathbf{p}) = \max_{\mathbf{q}\in\mathcal{N}(\mathbf{p})} S(\mathbf{q}) \qquad \text{dilatazione morfologica [15] su 9 campioni, raggio 14 px,} \tag{16}$$

$$M_s(\mathbf{p}) = \mathrm{S}(0{,}05,\ 0{,}3,\ S_d(\mathbf{p})) \qquad \text{soglia bassa e morbida,} \tag{17}$$

$$M(\mathbf{p}) = \max(M_s(\mathbf{p}),\ M_d(\mathbf{p})) \qquad \text{unione ("OR" sfumato) dei due indizi.} \tag{18}$$

Anche nel tempo vale lo stesso principio. La maschera della rete viene tenuta con un *peak hold* che decade lentamente,

$$S_t(\mathbf{p}) = \max\big(\hat{S}_t(\mathbf{p}),\ 0{,}8\, S_{t-1}(\mathbf{p})\big), \tag{19}$$

così un pixel appena lasciato dalla persona resta coperto per qualche fotogramma e i bordi non sfarfallano.

9.4 La transizione: sfocatura e dissolvenza

Un parametro $t \in [0, 1]$ va da 0 a 1 in 0,6 s (o al contrario per riapparire). Il colore "fantasma" G e l'uscita O sono

$$r(t) = 26\ \mathrm{S}(0,\ 0{,}55,\ t)\ \text{px}, \qquad w(t) = \mathrm{S}(0{,}3,\ 1,\ t), \tag{20}$$

$$G(\mathbf{p}) = (1 - w(t))\,\mathrm{Blur}_{r(t)}[I](\mathbf{p}) + w(t)\,B(\mathbf{p}), \tag{21}$$

$$O(\mathbf{p}) = (1 - M(\mathbf{p}))\,I(\mathbf{p}) + M(\mathbf{p})\,G(\mathbf{p}). \tag{22}$$

Nella prima metà la persona si sfoca rapidamente (r raggiunge quasi il massimo a metà transizione), poi si dissolve nello sfondo (w cresce da 0,3 in poi). A $t = 1$ resta solo B dentro la maschera, mentre fuori la scena è ancora dal vivo.

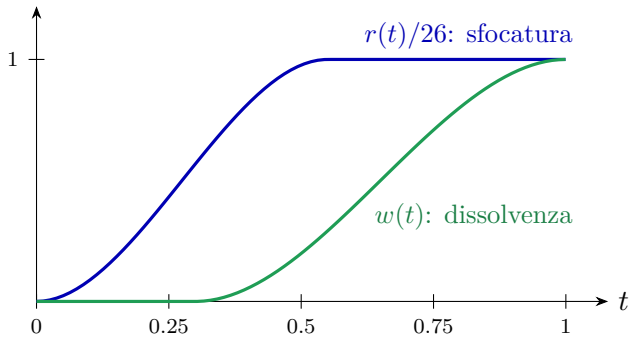


Figura 5: Le due curve della transizione.

Il cuore dello shader è breve:

```
float m = personMask(uv); // M: segmentation OR difference
vec3 bg = texture2D(uBg, uv).rgb; // B: clean plate
float radius = 26.0 * smoothstep(0.0, 0.55, t);
vec3 ghost = mix(blurAt(uv, radius), bg, smoothstep(0.3, 1.0, t));
col = mix(live, ghost, m); // O = (1-M) I + M G
```

t	$r(t)$	$w(t)$	dentro la maschera si vede
0	0 px	0	la persona nitida
0,25	11,2 px	0	la persona già sfocata
0,5	25,4 px	0,20	una sagoma molto sfocata che inizia a svanire
0,75	26 px	0,71	un fantasma trasparente
1	—	1	solo lo sfondo B

Tabella 2: Valori della transizione nei suoi punti chiave.

9.5 Limiti

- **Fotocamera mobile**: se l'inquadratura si sposta, B non è più allineata. Servirebbe registrare i fotogrammi, ad esempio stimando un'omografia con punti caratteristici, oppure ricostruire lo sfondo nel tempo.
- **Luce ed esposizione automatica**: se la luminosità cambia, D cresce ovunque. La maschera si allarga (senza conseguenze visibili) ma lo sfondo mostrato ha la luce "vecchia".
- **Zone mai viste**: la lastra contiene solo ciò che era visibile quando la scena era vuota. Per questo va catturata uscendo davvero dall'inquadratura.
- **Altre persone o oggetti in movimento** vengono trattati come primo piano dalla sottrazione dello sfondo e quindi nascosti anch'essi.

10 Tempo reale e privacy

A 30 fotogrammi al secondo ogni fotogramma ha un budget di circa 33 ms. Per rispettarlo l'app:

- analizza solo i fotogrammi nuovi e sfrutta il tracciamento di MediaPipe, che evita di rieseguire il rilevatore di palmi;
- esegue la segmentazione solo mentre la sparizione è attiva o in transizione;
- affida alla GPU tutto il lavoro per pixel. Il costo dipende soprattutto dal numero di letture di texture per pixel: 25 per la sfocatura, 8 per Sobel, una ventina per la maschera della persona;
- permette di ridurre la risoluzione di rendering dalle impostazioni sui dispositivi meno potenti.

Nessuna immagine lascia il dispositivo: modelli e librerie vengono scaricati una volta, poi l'inferenza e il compositing avvengono interamente nel browser.

Riferimenti bibliografici

[1] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, M. Grundmann. *MediaPipe Hands: On-device Real-time Hand Tracking.* arXiv:2006.10214, 2020.

[2] C. Lugaresi et al. *MediaPipe: A Framework for Building Perception Pipelines.* arXiv:1906.08172, 2019.

[3] V. Bazarevsky, Y. Kartynnik, A. Vakunov, K. Raveendran, M. Grundmann. *BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs.* arXiv:1907.05047, 2019.

[4] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, A. C. Berg. *SSD: Single Shot MultiBox Detector.* ECCV, 2016.

[5] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár. *Focal Loss for Dense Object Detection.* ICCV, 2017.

[6] A. Howard et al. *Searching for MobileNetV3.* ICCV, 2019.

[7] G. Casiez, N. Roussel, D. Vogel. *1 € Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems.* CHI, 2012.

[8] I. Quilez. *2D distance functions.* `https://iquilezles.org/articles/distfunctions2d/`.

[9] T. Porter, T. Duff. *Compositing Digital Images.* SIGGRAPH, 1984.

[10] ITU-R. *Recommendation BT.601: Studio encoding parameters of digital television.*

[11] I. Sobel, G. Feldman. *A 3×3 Isotropic Gradient Operator for Image Processing.* Stanford Artificial Intelligence Project, 1968.

[12] R. C. Gonzalez, R. E. Woods. *Digital Image Processing*, 4ª ed. Pearson, 2018.

[13] H. Vogel. *A better way to construct the sunflower head.* Mathematical Biosciences, 44(3–4), 1979.

[14] M. Piccardi. *Background subtraction techniques: a review.* IEEE International Conference on Systems, Man and Cybernetics, 2004.

[15] J. Serra. *Image Analysis and Mathematical Morphology.* Academic Press, 1982.